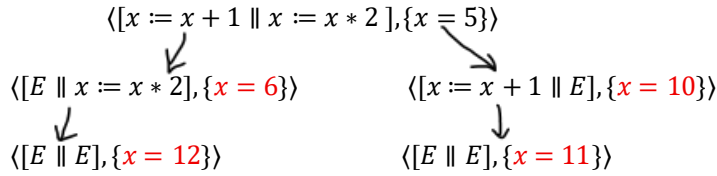


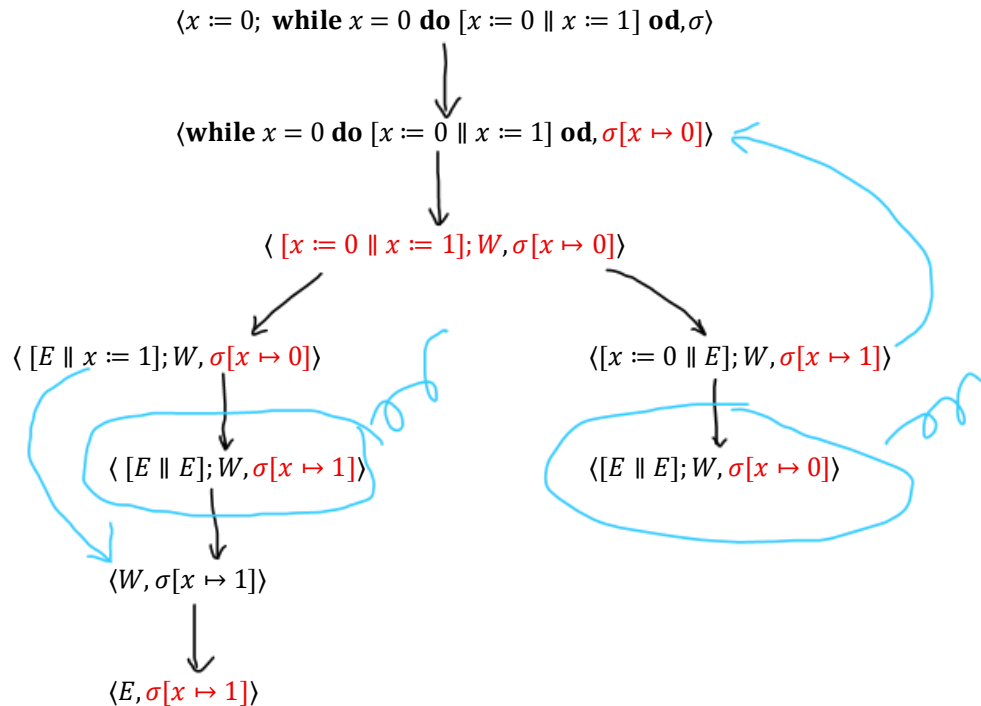
Parallel Program Basics

- A parallel program is trying to run all different threads “at the same time”. In our language, the syntax of a parallel statement/program with n threads is $S \equiv [S_1 \parallel S_2 \parallel \dots \parallel S_n]$. We say $[S_1 \parallel S_2 \parallel \dots \parallel S_n]$ is the **parallel composition of threads** $S_1, S_2, \dots, S_n$.
 - Each thread S_i in the composition should be non-parallel and deterministic: it is not legal to write $S \equiv [S_1 \parallel [S_2 \parallel S_3]]$.
- Before we formally define the semantics of parallel programs, let’s use a simple example to see the difference between sequential, parallel, and nondeterministic conditional programs.
 1. Find a postcondition for each of the following valid triples.
 - a) $\{x = 5\} x := x + 1; x := x * 2 \{q\}$
 It is quite easy to see that $x = 12$ is a valid postcondition: we will finish two assignments in the given order. It is almost the strongest postcondition, we only omitted the initial value of x compared to $x_0 = 5 \wedge x_1 = x_0 + 1 \wedge x = x_1 * 2$.
 - b) $\{x = 5\} \text{if } T \rightarrow x := x + 1 \square T \rightarrow x := x * 2 \text{fi } \{q\}$
 Both arms have true guard, so we will execute two branches at the same time with equal probability. Thus, the postcondition is $x = 6 \vee x = 10$. As an aside, the strongest postcondition is $x_0 = 5 \wedge x = x_0 + 1 \vee x_0 = 5 \wedge x = x_0 * 2$.
 - c) $\{x = 5\} [x := x + 1 \parallel x := x * 2] \{q\}$
 Both threads will be executed “at the same time”; but some thread must be executed faster than the other in real life, and threads will be executed in any possible order. Thus, we might have $x = 12$ if we execute $x := x + 1$ first, or we might have $x = 11$ if we execute $x := x * 2$ first. Thus, $x = 11 \vee x = 12$ is a valid postcondition here.
- The above example shows the difference between sequential, parallel, and nondeterministic programs.
 - For a sequential statement, we execute each unit statement in the given order.
 - For a nondeterministic **if – fi** statement, we execute each arm at the same time with the same probability.
 - For a parallel statement, all unit statements in the composition will be executed in any possible order. So, parallel statements can be considered as a **simulation of nondeterminism**: $[x := x + 1 \parallel x := x * 2]$ can simulate **if** $T \rightarrow x := x + 1; x := x * 2 \square T \rightarrow x := x * 2; x := x + 1$ **fi**.
- **Operational semantic** of parallel statements: given $S \equiv [S_1 \parallel S_2 \parallel \dots \parallel S_n]$, for each $k = 1, 2, \dots, n$, if $\langle S_k, \sigma \rangle \rightarrow \langle T_k, \tau_k \rangle$, then $\langle [S_1 \parallel \dots \parallel S_{k-1} \parallel S_k \parallel S_{k+1} \parallel \dots \parallel S_n], \sigma \rangle \rightarrow \langle [S_1 \parallel \dots \parallel S_{k-1} \parallel T_k \parallel S_{k+1} \parallel \dots \parallel S_n], \tau_k \rangle$. If we don’t have any runtime error or divergence, the execution of S will end with configuration $\langle E \equiv [E \parallel E \parallel \dots \parallel E], \tau \rangle$.
 - Note that, from each configuration, we can go to at most n configurations in the next step.
 - The notations $\rightarrow^*$ and $\rightarrow^k$ that we used in a non-parallel program still work here.
 - Note that, $E \equiv [E \parallel E \parallel \dots \parallel E]$. It is a common mistake to write $\langle [E \parallel E \parallel \dots \parallel E], \tau \rangle \rightarrow \langle E, \tau \rangle$; we can write $\langle [E \parallel E \parallel \dots \parallel E], \tau \rangle \rightarrow^0 \langle E, \tau \rangle$ since $E \equiv [E \parallel E \parallel \dots \parallel E]$.
- Remember that **denotational semantics** of a statement in a state is the collection of all possible terminating (possibly pseudo-) states. The definition is the same for parallel statements.

2. Show the operational semantic for $\langle S, \sigma \rangle$ till the end where $S \equiv [x := x + 1 \parallel x := x * 2]$ and $\sigma = \{x = 5\}$. What is $M(S, \sigma)$?



- $M(S, \sigma) = \{\{x = 12\}, \{x = 11\}\}$. Since parallel statements can be considered as a simulation of nondeterminism, it is still true that "If $M(S, \sigma)$ contains more than one states, then S is nondeterministic."
- In a parallel program, at each configuration there is usually more than one configuration that can be the next step, so the analysis of operational semantics usually gives directed graph instead of a list, we call this graph an **evaluation graph**.
 - While drawing an evaluation graph, we need to make sure that:
 - 1) Each vertex in the graph is a configuration, and each configuration is *unique*.
 - 2) Each directed edge shows one step (or n steps if $\rightarrow^n$, or any number steps if $\rightarrow^*$) in the evaluation, and we don't allow multi-edge in the graph. In other words, if we go from one configuration to another twice, we only draw this edge once in the graph.
 - 3) All the possible executions need to be shown in the graph: if a thread composition has n threads, then there can be at most n outgoing edges from that node.
- 3. Let $W \equiv \text{while } x = 0 \text{ do } [x := 0 \parallel x := 1] \text{ od}$. Draw evaluation graph for $\langle x := 0; W, \sigma \rangle$, and calculate $M(x := 0; W, \sigma)$.



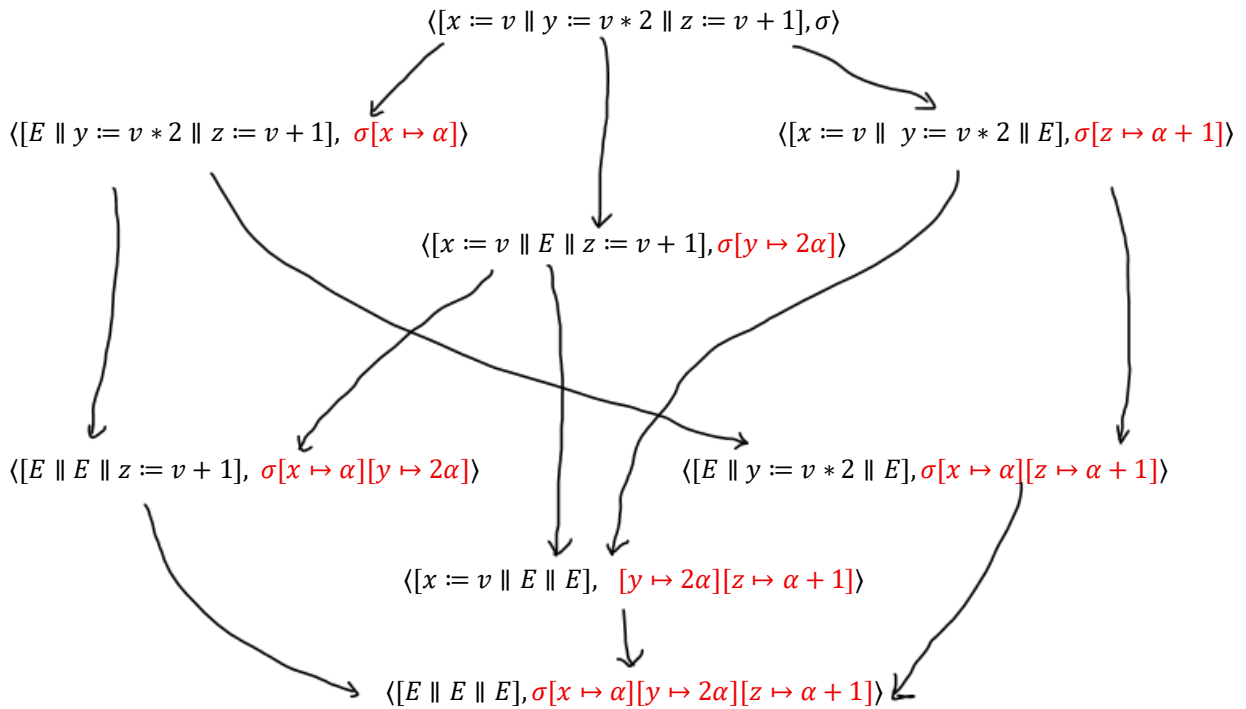
- From the evaluation graph, we can see that $M(S, \sigma) = \{\perp_d, \sigma[x \mapsto 1]\}$
- We can see that W might diverge on σ : in each iteration if we execute $x := 1$ first, then we will end up with $x = 0$ and we will go to another iteration.

Race Conditions

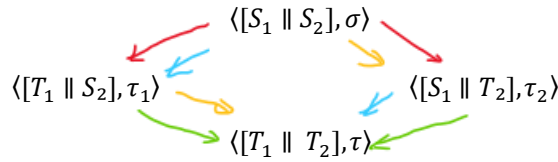
- Let program S contains parallel statement $[S_1 \parallel S_2 \parallel \dots \parallel S_n]$ and $\not\models_{tot} \{p\} S \{q\}$; if this invalidity is caused by the *execution order (or relative speed)* of threads in $[S_1 \parallel S_2 \parallel \dots \parallel S_n]$, then we say S has **race conditions**.
4. Does each of the following triples have race conditions?
- a) $\{T\}[x := 0 \parallel x := 1] \{x > 0\}$
 Yes. We can get $x = 0$ if we execute the first thread after the second thread, it doesn't satisfy the postcondition.
- b) $\{T\}[x := 0 \parallel x := 1] \{x \geq 0\}$
 No. The program can end up with $x = 0$ or 1 , they both satisfy the postcondition.
- c) $\{T\}[x := 0 \parallel x := 1]; z := 0 \div x \{z = 0\}$
 Yes. If we execute the second thread first then we will end up with a runtime error.
- d) $\{T\}[x := 0 \parallel x := 1] \{x > 2\}$
 No. The execution order is not the reason for the invalidity.

Disjoint Parallel Programs

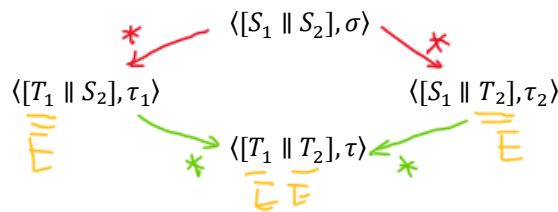
5. Draw the evaluation graph for configuration $\langle [x := v \parallel y := v * 2 \parallel z := v + 1], \sigma \rangle$ where $\sigma(v) = \alpha$, and v, x, y, z are different variables.



- In this example, although the program is parallel, it generates the same state no matter what execution paths it uses.
- The program in the above example is a **disjoint parallel program**. Disjoint parallel programs model the situation that multiple threads share readable memory but not writable memory. For every variable x in a disjoint parallel program, there are two situations:
 - a) All threads can read x and no threads can write x . In example 5, every thread reads v and no thread writes v .
 - b) At most 1 thread can read and write x , and other threads can neither read nor write x . In example 5, the first thread writes x so other threads can *neither read nor write* x .
- In general, with $[S_1 \parallel S_2]$ we can execute either S_1 or S_2 for the next step. In an evaluation graph, the current evaluation path splits into two paths. In general, there might be no way for those two paths to eventually merge back together into one path, but disjoint parallel programs are different.
- **[Diamond Property]** Let $[S_1 \parallel S_2]$ be a disjoint parallel program. If $\langle S_1, \sigma \rangle \rightarrow \langle T_1, \tau_1 \rangle$ and $\langle S_2, \sigma \rangle \rightarrow \langle T_2, \tau_2 \rangle$ then there is a state τ such that $\langle [T_1 \parallel S_2], \tau_1 \rangle$ and $\langle [S_1 \parallel T_2], \tau_2 \rangle$ both $\rightarrow \langle [T_1 \parallel T_2], \tau \rangle$.
 - It is easy to see the property is true since the order of execution of S_1 and S_2 does not matter: any change in state caused by the S_1 will be the same despite whether we execute S_2 , since S_1 and S_2 write into different variables and they don't read the variables the other thread writes (and vice-versa).
 - It is also easy to see how the property is named if we draw the evaluation graph. Here I use different colors on directed edges: it means if red edges exist, then the green edges exist.



- Diamond property can be generalized into a more general property called **confluence**: where the one-step arrows ($\rightarrow$ becomes $\rightarrow^*$). Similarly, if red edges exist, then green edges exist.



- **[Confluence]** Let $[S_1 \parallel S_2]$ be a disjoint parallel program. If $\langle S_1, \sigma \rangle \rightarrow^* \langle T_1, \tau_1 \rangle$ and $\langle S_2, \sigma \rangle \rightarrow^* \langle T_2, \tau_2 \rangle$ then there is a state τ such that $\langle [T_1 \parallel S_2], \tau_1 \rangle$ and $\langle [S_1 \parallel T_2], \tau_2 \rangle$ both $\rightarrow^* \langle [T_1 \parallel T_2], \tau \rangle$.
 - It is easy to see that the diamond property implies confluence (confluence is a generalized diamond property).
- Because execution of disjoint parallel programs is confluent, if execution terminates, it terminates in a unique state. This gives us the following theorem.
- **[Theorem (Unique Result of Disjoint Parallel Program)]**: If S is a disjoint parallel program then $M(S, \sigma) = \{\tau\}$, where $\tau \in \Sigma \cup \{\perp_d, \perp_e\}$.
 - It is easy to prove this theorem using confluence: If $\langle S_1, \sigma \rangle \rightarrow^* \langle E, \tau_1 \rangle$ and $\langle S_2, \sigma \rangle \rightarrow^* \langle E, \tau_2 \rangle$ then there is a state τ such that $\langle [E \parallel S_2], \tau_1 \rangle$ and $\langle [S_1 \parallel E], \tau_2 \rangle$ both $\rightarrow^* \langle [E \parallel E], \tau \rangle$.